

INFORMATION RETRIEVAL

PROJECT REPORT

TF-IDF based Document Search Engine
with Cosine Similarity Ranking

Course: Information Retrieval (IR)

Assignment: TF-IDF Search Engine Implementation

Domain: Technology, Environment & Education

Dataset: 20 Documents (soft copy in 'documents_dataset/' folder)

Submitted By: Student

Date of Submission: August 2026

Libraries Used: NLTK, Scikit-learn, Pandas, NumPy, ReportLab

1. Introduction

This assignment implements a document search engine using the TF-IDF methodology. The program accepts a user query, searches through a corpus of documents, counts how many times query terms appear in each document, computes a TF-IDF weighted score, and returns a ranked list sorted in descending order, where Rank 1 corresponds to the highest score.

The corpus consists of 20 documents saved as individual .txt text files in the `documents_dataset/` folder, which are loaded into the program at runtime. The implementation uses Python with NLTK for stop-word removal and Porter stemming, Scikit-learn for TF-IDF vectorization and cosine similarity computation, and Pandas for structured result display.

Input → Process → Output Flow:

- **Input:** A corpus of 20 text documents loaded from folder + a user search query.
- **Process:** Six-step text preprocessing → TF-IDF matrix construction → query vectorization → per-term occurrence counting → cosine similarity scoring.
- **Output:** Ranked results in descending order, showing Rank, Doc ID, Total matches, per-Term counts, and final TF-IDF score.

2. Dataset

Each document is saved as an individual .txt file in the `documents_dataset/` folder. The program scans the folder, reads all .txt files, and constructs the in-memory document dictionary. This design allows the corpus to be modified simply by adding or editing text files, without touching the source code.

Document List:

1. `Doc_1.txt` (25 words) — Artificial intelligence refers to computer systems that can perform tasks normally requiring human intelligence, including visual perception, speech recognition, decision making, and natural language translation.
2. `Doc_10.txt` (24 words) — Online learning delivers education through digital platforms, video lectures, interactive quizzes, and virtual classrooms, enabling students to study from anywhere at their own pace.
3. `Doc_11.txt` (29 words) — Blockchain is a decentralized and distributed digital ledger that records transactions across many computers so that no single participant can alter the record retroactively without changing all subsequent blocks.
4. `Doc_12.txt` (25 words) — Water pollution occurs when harmful substances like chemicals, plastics, or sewage contaminate rivers, lakes, oceans, and groundwater, damaging aquatic ecosystems and human drinking water supplies.
5. `Doc_13.txt` (25 words) — Robotics combines mechanical engineering, electronic engineering, and computer science to design, construct, operate, and use robots that can assist humans in manufacturing, healthcare, and exploration.
6. `Doc_14.txt` (30 words) — Wind energy uses large turbines mounted on towers to capture the kinetic energy of moving air and convert it into electrical power, producing zero fuel cost or greenhouse gas emissions.
7. `Doc_15.txt` (25 words) — Natural language processing is a branch of artificial intelligence that helps computers understand, interpret, and generate human language, powering chatbots, translation tools, and voice assistants.
8. `Doc_16.txt` (23 words) — Electric vehicles use rechargeable battery packs to power electric motors instead of internal combustion engines, reducing transportation emissions and dependence on petroleum-based gasoline.
9. `Doc_17.txt` (24 words) — Computer vision enables machines to extract meaningful information from digital images and videos, enabling applications like facial recognition, autonomous vehicles, and medical image analysis.
10. `Doc_18.txt` (28 words) — Biodiversity refers to the variety of all living species on Earth, including plants, animals, bacteria, and fungi, and it supports ecosystem stability, food security, and natural medicine discovery.
11. `Doc_19.txt` (26 words) — Big data analytics examines extremely large and varied datasets to uncover hidden patterns, market trends, customer preferences, and other useful business insights using

specialized software tools.

12. Doc_2.txt (25 words) — Cloud computing delivers on-demand computing resources over the internet, such as servers, storage, databases, networking, software, and analytics, without direct active management by the user.

13. Doc_20.txt (25 words) — Digital libraries organize and provide access to vast collections of electronic books, journals, research papers, and multimedia resources, enabling remote learning and global knowledge sharing.

14. Doc_3.txt (26 words) — Climate change describes long-term shifts in global temperatures and weather patterns, mainly driven by human activities like burning fossil fuels, deforestation, and industrial greenhouse gas emissions.

15. Doc_4.txt (28 words) — Machine learning is a subset of artificial intelligence that enables systems to learn patterns from data and improve their performance on a specific task without being explicitly programmed.

16. Doc_5.txt (28 words) — Renewable energy comes from naturally replenishing sources such as sunlight, wind, rain, tides, waves, and geothermal heat, offering a cleaner alternative to fossil fuels like coal and oil.

17. Doc_6.txt (27 words) — The Internet of Things connects everyday physical objects embedded with sensors, software, and network connectivity, enabling them to collect and exchange data across smart homes and cities.

18. Doc_7.txt (24 words) — Deforestation involves the permanent removal of trees to make room for agriculture, grazing, or urban expansion, threatening biodiversity and accelerating global carbon dioxide levels.

19. Doc_8.txt (22 words) — Cybersecurity protects computer systems, networks, programs, and data from digital attacks, damage, or unauthorized access through encryption, firewalls, and intrusion detection systems.

20. Doc_9.txt (24 words) — Solar power generates electricity by converting sunlight using photovoltaic panels or concentrated solar power, providing a sustainable and increasingly affordable source of renewable energy.

Code for loading documents from folder:

```
import os

DATASET_FOLDER = os.path.join(os.path.dirname(__file__), 'documents_dataset')

def load_documents_from_folder(folder=DATASET_FOLDER):
    docs = {}
    if os.path.isdir(folder):
        for fname in sorted(os.listdir(folder)):
            if fname.lower().endswith('.txt'):
                name = os.path.splitext(fname)[0].replace('_', ' ')
                with open(os.path.join(folder, fname), 'r') as f:
                    text = f.read().strip()
```

```
        if text:
            docs[name] = text
    return docs

documents = load_documents_from_folder()
print(f'Loaded {len(documents)} documents from disk.')
```

3. Text Preprocessing

Every document and every query passes through the same six-step cleaning pipeline, applied in the order listed below.

- **Step 1 — Lowercase:** Convert every character to lowercase.
- **Step 2 — Remove punctuation:** Strip non-alphabetic characters with regex.
- **Step 3 — Tokenize:** Split the cleaned string on whitespace into tokens.
- **Step 4 — Remove stop words:** Drop NLTK English stop words (the, and, for, is, ...).
- **Step 5 — Drop short tokens:** Discard tokens with length 1 or less.
- **Step 6 — Porter stemming:** Reduce each token to its morphological root.

Preprocessing code:

```
import re, nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

nltk.download('stopwords', quiet=True)
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()

def preprocess_text(text):
    text = re.sub(r'^a-zA-Z\s]', '', text.lower())
    tokens = text.split()
    cleaned = []
    for word in tokens:
        if word not in stop_words and len(word) > 1:
            cleaned.append(stemmer.stem(word))
    return ' '.join(cleaned)
```

Sample output (first 4 documents):

```
--- SAMPLE PREPROCESSING OUTPUT ---

Doc 1 (Original): Artificial intelligence refers to computer systems that can perform
tasks normally requiring hu...
Doc 1 (Cleaned) : artifici intellig refer comput system perform task normal requir hum
an intellig includ visual percept speech recognit decis make natur languag translat

Doc 10 (Original): Online learning delivers education through digital platforms, video
lectures, interactive quizz...
Doc 10 (Cleaned) : onlin learn deliv educ digit platform video lectur interact quizz v
irtual classroom enabl student studi anywher pace

Doc 11 (Original): Blockchain is a decentralized and distributed digital ledger that r
ecords transactions across m...
Doc 11 (Cleaned) : blockchain decentr distribut digit ledger record transact across ma
```

ni comput singl particip alter record retroact without chang subsequ block

Doc 12 (Original): Water pollution occurs when harmful substances like chemicals, plastics, or sewage contaminate ...

Doc 12 (Cleaned) : water pollut occur harm substanc like chemic plastic sewage contaminate
river lake ocean groundwat damag aquat ecosystem human drink water suppli

4. TF-IDF Feature Matrix

Definitions:

- **TF (Term Frequency):** Number of times term t appears in document d , divided by the total number of tokens in d . Normalizes for document length.
- **IDF (Inverse Document Frequency):** $IDF(t) = \ln(N / df(t)) + 1$, where N is the total number of documents and $df(t)$ is how many documents contain term t . Rare terms receive higher IDF weights.
- **TF-IDF weight:** Element-wise product $TF(t,d) \times IDF(t)$. Terms that are rare in the corpus but frequent within a document receive the highest weights.

Matrix construction code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

processed_corpus = {did: preprocess_text(txt) for did, txt in documents.items()}

vectorizer = TfidfVectorizer()
corpus_list = list(processed_corpus.values())
doc_ids     = list(processed_corpus.keys())

tfidf_matrix = vectorizer.fit_transform(corpus_list)
feature_names = vectorizer.get_feature_names_out()

tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(), index=doc_ids, columns=feature_names
)
```

Matrix summary output:

```
--- TF-IDF FEATURE MATRIX SUMMARY ---

Total documents (corpus size) : 20
Unique vocabulary terms (V)   : 271
TF-IDF matrix dimensions      : 20 rows x 271 columns
Non-zero entries in matrix     : 371
Matrix sparsity                : 93.15 %
```


5. Query Processing and Ranking

The query is preprocessed by the same six-step pipeline, then converted into a vector using `vectorizer.transform()`. Critically, `fit_transform()` is never used on the query, as doing so would re-learn the vocabulary and break column alignment with the document matrix.

Search and ranking code:

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def raw_term_counts(doc_text, query_terms):
    tokens = re.sub(r'^a-zA-Z\s', '', doc_text.lower()).split()
    stems = [stemmer.stem(t) for t in tokens if len(t) > 1]
    per_term = {qt: stems.count(qt) for qt in query_terms}
    return per_term, sum(per_term.values())

def search_and_rank(user_query, top_k=10):
    processed_q = preprocess_text(user_query)
    query_tokens = processed_q.split()
    query_vector = vectorizer.transform([processed_q])
    scores = cosine_similarity(query_vector, tfidf_matrix).flatten()
    ranked = np.argsort(scores)[::-1]

    print(f'USER QUERY          : {user_query}')
    print(f'PREPROCESSED QUERY      : {processed_q}')
    for rank, idx in enumerate(ranked[:top_k], 1):
        did = doc_ids[idx]
        per, tot = raw_term_counts(documents[doc_ids[idx]], query_tokens)
        print(f'{rank:<5}{did:<8}{tot:<7} ... {scores[idx]:.4f}')
```

Cosine similarity, rather than Euclidean distance, is used so that short and long documents on the same topic still produce high similarity scores. The formula is $\cos(d, q) = (d \cdot q) / (||d|| \times ||q||)$, with a range of 0.0 to 1.0; higher values indicate greater similarity.

Query 1 — Artificial Intelligence & Machine Learning

```
=====
USER QUERY          : artificial intelligence machine learning data
PREPROCESSED QUERY  : artifici intellig machin learn data
QUERY TERMS (STEMMED): ['artifici', 'intellig', 'machin', 'learn', 'data']
Rank Doc ID  Total  Term Counts                                TF-IDF Score
-----
1   Doc 4    6      artifici:1, intellig:1, machin:1, learn:2, data:10.5769
2   Doc 1    3      artifici:1, intellig:2, machin:0, learn:0, data:00.2543
3   Doc 15   2      artifici:1, intellig:1, machin:0, learn:0, data:00.1746
4   Doc 17   1      artifici:0, intellig:0, machin:1, learn:0, data:00.0987
5   Doc 10   1      artifici:0, intellig:0, machin:0, learn:1, data:00.0907
```

Doc 4 is ranked first because it contains the stems *artifici*, *intellig*, *machin* with a TF-IDF score of 0.5769. Other documents share fewer stems and consequently receive lower scores.

Query 2 — Renewable Energy

```
=====
USER QUERY           : renewable energy solar wind power environment
PREPROCESSED QUERY   : renew energi solar wind power environ
QUERY TERMS (STEMMED): ['renew', 'energi', 'solar', 'wind', 'power', 'environ']
Rank Doc ID  Total  Term Counts                                     TF-IDF Score
-----
1   Doc 9    6      renew:1, energi:1, solar:2, wind:0, power:2, environ:00.5392
2   Doc 14   4      renew:0, energi:2, solar:0, wind:1, power:1, environ:00.3031
3   Doc 5    3      renew:1, energi:1, solar:0, wind:1, power:0, environ:00.2651
4   Doc 15   1      renew:0, energi:0, solar:0, wind:0, power:1, environ:00.0680
5   Doc 16   1      renew:0, energi:0, solar:0, wind:0, power:1, environ:00.0653
```

Doc 9 and **Doc 14** occupy the top two positions since they directly discuss solar and wind energy, both of which match the renewable energy theme of the query.

Query 3 — Cloud Computing & Cybersecurity

```
=====
USER QUERY           : cloud computing cybersecurity data network
PREPROCESSED QUERY   : cloud comput cybersecur data network
QUERY TERMS (STEMMED): ['cloud', 'comput', 'cybersecur', 'data', 'network']
Rank Doc ID  Total  Term Counts                                     TF-IDF Score
-----
1   Doc 8    4      cloud:0, comput:1, cybersecur:1, data:1, network:10.3476
2   Doc 2    4      cloud:1, comput:2, cybersecur:0, data:0, network:10.3184
3   Doc 6    2      cloud:0, comput:0, cybersecur:0, data:1, network:10.1436
4   Doc 4    1      cloud:0, comput:0, cybersecur:0, data:1, network:00.0761
5   Doc 19   1      cloud:0, comput:0, cybersecur:0, data:1, network:00.0632
```

Doc 8 receives Rank 1 because it contains the terms cloud, computing, servers, storage, and networking, matching most of the query stems. Cybersecurity-related terms boost the next-highest documents.

Query 4 — Education & Digital Libraries

```
=====
USER QUERY           : education online learning digital libraries books
PREPROCESSED QUERY   : educ onlin learn digit librari book
QUERY TERMS (STEMMED): ['educ', 'onlin', 'learn', 'digit', 'librari', 'book']
Rank Doc ID  Total  Term Counts                                     TF-IDF Score
-----
1   Doc 10   4      educ:1, onlin:1, learn:1, digit:1, librari:0, book:00.3520
2   Doc 20   4      educ:0, onlin:0, learn:1, digit:1, librari:1, book:10.3313
3   Doc 4    2      educ:0, onlin:0, learn:2, digit:0, librari:0, book:00.1506
4   Doc 8    1      educ:0, onlin:0, learn:0, digit:1, librari:0, book:00.0521
5   Doc 11   1      educ:0, onlin:0, learn:0, digit:1, librari:0, book:00.0460
```

Doc 10 (online learning) and **Doc 20** (digital libraries) are the top-ranked documents, as expected, because they directly address the educational and library-related terms in the query.

6. Pipeline Evaluation

Stop-word removal and stemming compress the vocabulary, which in turn reduces the TF-IDF matrix dimensions and speeds up query processing. Actual metrics from the current run are reported below.

```
def evaluate_pipeline():
    raw_words = re.sub(r'^a-zA-Z\s|', '', ' '.join(documents.values()).lower()).split()
    processed = ' '.join(processed_corpus.values()).split()
    unique_raw = set(raw_words)
    processed_v = set(feature_names)
    reduction = ((len(unique_raw) - len(processed_v)) / len(unique_raw)) * 100
    print('Total raw word count (before cleaning) :', len(raw_words))
    print('Total stemmed tokens (after cleaning) :', len(processed))
    print('Unique raw vocabulary (before) :', len(unique_raw))
    print('Unique TF-IDF features (after) :', len(processed_v))
    print('Net vocabulary reduction :', f'{reduction:.2f} %')
```

Evaluation output:

```
--- PIPELINE EVALUATION METRICS ---

Total raw word count (before cleaning) : 513
Total stemmed tokens (after cleaning) : 388
Unique raw vocabulary (before) : 338
Unique TF-IDF features (after) : 271
Net vocabulary reduction : 19.82 %
```

The vocabulary shrinks by approximately 20% after preprocessing. Most of the reduction comes from stop-word removal; a smaller fraction comes from merging morphological variants such as `compute/computing/computed` into a single stem.

7. IRSystem Class

The full search engine is also encapsulated in a reusable Python class. The advantage is that the `TfidfVectorizer` is fitted exactly once during object construction. After that, the `search()` method can be called repeatedly on different queries without re-fitting the vectorizer or rebuilding the matrix.

Class members:

- `__init__(doc_dict)` — Constructor. Saves raw documents, creates stemmer and stop-word set, preprocesses all documents, and fits the TF-IDF vectorizer.
- `_preprocess(text)` — Private helper. Runs the six-step cleaning pipeline on a single string.

- **search(query, top_k)** — Public API. Preprocesses the query, vectorizes it, counts per-term matches per document, computes cosine similarity, and returns a DataFrame of top_k results in descending order.
- **self.tfidf_matrix** — Fitted SciPy CSR sparse matrix (rows = documents, columns = vocabulary terms).
- **self.feature_names** — NumPy array of vocabulary terms in column order.

Full class code:

```
class IRSystem:
    def __init__(self, doc_dict):
        self.raw_documents = doc_dict
        self.stemmer = PorterStemmer()
        self.stop_words = set(stopwords.words('english'))
        self.vectorizer = TfidfVectorizer()
        self.processed_corpus = {k: self._preprocess(v) for k, v in self.raw_documents.items()}
        self.doc_ids = list(self.processed_corpus.keys())
        self.tfidf_matrix = self.vectorizer.fit_transform(list(self.processed_corpus.values()))
        self.feature_names = self.vectorizer.get_feature_names_out()

    def _preprocess(self, text):
        text = re.sub(r'^[a-zA-Z\s]', '', text.lower())
        cleaned = []
        for w in text.split():
            if w not in self.stop_words and len(w) > 1:
                cleaned.append(self.stemmer.stem(w))
        return ' '.join(cleaned)

    def search(self, query, top_k=5):
        pv = self._preprocess(query)
        q_tokens = pv.split()
        q_vec = self.vectorizer.transform([pv])
        scores = cosine_similarity(q_vec, self.tfidf_matrix).flatten()
        ranked_indices = np.argsort(scores)[::-1]
        results = []
        for rank, idx in enumerate(ranked_indices[:top_k], 1):
            per_term, total = self._raw_counts(self.raw_documents[self.doc_ids[idx]], q_tokens)
            results.append({'Rank': rank, 'Doc ID': self.doc_ids[idx],
                           'Total Matches': total, 'Term Counts': per_term,
                           'TF-IDF Score': round(scores[idx], 4)})
        return pd.DataFrame(results)
```

Class demonstration output:

```
--- IRSYSTEM CLASS DEMONSTRATION ---
```

```
Example: ir.search('blockchain decentralized ledger transactions', top_k=3)
```

Rank	Doc ID	Total	Term Counts	TF-IDF Score
1	Doc 11	4	blockchain:1, decentr:1, ledger:1, transact:1	10.4592
2	Doc 9	0	blockchain:0, decentr:0, ledger:0, transact:0	00.0000
3	Doc 7	0	blockchain:0, decentr:0, ledger:0, transact:0	00.0000

The top-ranked document for the blockchain query is Doc 11, which directly defines blockchain as a decentralized distributed ledger recording transactions across multiple computers.

8. Conclusion

- Preprocessing step order is significant. Punctuation must be removed before stop-word removal and stemming, otherwise malformed tokens can leak into the vocabulary.
- Stop words should be removed before stemming to avoid wasting computation on words that will be discarded anyway.
- For queries, `transform()` must be used rather than `fit_transform()`. Re-fitting re-learns the vocabulary and produces meaningless scores.
- Displaying raw term occurrence counts alongside TF-IDF scores makes the ranking explainable; the user can clearly see which terms matched in each document and why a document received a particular rank.
- Ranking is strictly descending: the document with the highest TF-IDF cosine similarity score is assigned Rank 1, the next-highest receives Rank 2, and so on. Documents sharing no query terms correctly receive a score of zero.
- Using a separate soft-copy dataset folder (`documents_dataset/*.txt`) makes it easy to swap the corpus: new .txt files can be added or existing files edited without modifying the source code.
- Encapsulating the engine in the `IRSystem` class keeps repeated queries efficient because the TF-IDF matrix is constructed exactly once, at instantiation time.

9. Files Submitted

- **`ir_project_code.py`** — Main Python script. Loads 20 documents from the folder, runs preprocessing, builds the TF-IDF matrix, executes four hard-coded test queries with full term-count and score output, runs the evaluation, demonstrates the `IRSystem` class, and finally enters an interactive query mode.
- **`tfidf_matrix_numerical.py`** — Stand-alone interactive script for manual step-by-step TF-IDF calculation. Shows per-document TF, per-term IDF, element-wise TF-IDF matrix, per-term occurrence counts, and final descending ranking.
- **`documents_dataset/`** — Folder containing the soft-copy corpus as 20 individual text files (`Doc_1.txt` through `Doc_20.txt`). The program reads these files at startup.
- **`generate_pdf.py`** — Python script used to programmatically generate this project report PDF, using the ReportLab library.
- **`IR_Project_Report.pdf`** — This report in PDF form.